CETP quickstart

Predict how long a PostgreSQL query will take on a production machine, using a plan captured on your development machine.

Install

```
pip install -e .
cetp selftest
```

`selftest` checks redaction, prediction, regression detection and the calibration table end to end. If it fails, nothing below will work.

Predicting production runtime (the four-move flow)

This is the path toward the project's actual goal, and it replaces the two-scalar hardware signature — which, on the one experiment that tested the real claim (arm64 laptop -> x86 EC2), scored **324% MAPE against 123% for assuming production equals development**. It was worse than doing nothing. Design and status: OPERATOR_MODEL.md.

```
# 1. register both machines. SQL only -- so this works on RDS/Aurora/Cloud SQL,
#    which the old C probe could not touch. Run it on an IDLE machine: the probe
#    refuses itself if the machine is busy, because then it measures contention.
python3 scripts/collection/operator_probe.py --machine-id dev-laptop --db mydb
python3 scripts/collection/operator_probe.py --machine-id prod-db --db mydb \
    --host prod.example.com --user readonly

# 2. predict. Production is only EXPLAINed (never executed); its plan is then forced
#    on dev and measured there.
python3 scripts/collection/predict_prod.py --query-file q.sql \
    --dev dev-laptop --prod prod-db \
    --prod-host prod.example.com --prod-db mydb --prod-user readonly \
    --dev-db mydb --sla-ms 2000
```

Or from a pair of plan files you already have:

```
cetp predict-prod --dev-plan dev.json --prod-plan prod.json \
    --dev dev-laptop --prod prod-db --plan-forced yes
```

What makes this different from `cetp predict`: capability is six measured operator rates rather than two correlated scalars; the prediction is weighted per operator, so it can express "fast at scans, slow at sorts"; the plan comes from production, so plan divergence (**70.5%** of the cross-environment difference when the plan changes) is removed by construction rather than modelled.

What it still does not do: concurrency. Every number is single-client, and measured elasticity reaches **8.57× at 8 clients** — larger than every term above. Supply your production load yourself; it cannot be inferred from a development machine. The accuracy claim is also **not yet established**: see the status table in OPERATOR_MODEL.md.

The recommended mode for today: regression detection

Use this one. Absolute prediction carries about 21.6% MAPE on a machine the model has never seen, which is not accurate enough to gate an SLA — in the research evaluation 54.5% of absolute BLOCK verdicts were false alarms.

Regression detection compares the *same* query before and after a change, so the hardware term appears in both the numerator and the denominator and largely cancels:

```
predicted_prod_after     dev_after      s_after
--------------------  =  ---------  x  ---------
predicted_prod_before    dev_before     s_before
```

When `s_after / s_before` is near 1.0 the answer is essentially your directly measured dev-side ratio, and the cross-machine model is barely involved.

```
# capture the plan before and after your change, on the same dev machine
psql -d mydb -tAc "EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) $(cat before.sql)" > before.json
psql -d mydb -tAc "EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) $(cat after.sql)"  > after.json

cetp regress --before before.json --after after.json \
             --dev c7i --prod c5a --query-id q7 --threshold 1.20
```

```
CETP regression check for q7: REGRESSION
  dev runtime:        412.0 ms -> 988.0 ms  (2.40x)
  predicted prod:     244.0 ms -> 585.0 ms  (2.40x)
  hardware term:      1.000x  (cancelled)
  plan nodes:         7 -> 11
  trust:              HIGH: the hardware term cancelled (within 2%), so this ratio is
                      essentially the directly measured dev-side ratio and does not depend
                      on the cross-machine model being right.
```

Add `--fail-on-regression` to exit non-zero, which is all CI needs.

Absolute prediction, with its error stated

```
cetp predict --plan after.json --dev c7i --prod c5a --sla-ms 3000
```

Reports the point estimate plus calibrated upper bounds (90/95/99%), a confidence score, and every warning that applies. **Read the warnings.** Coverage of the bounds is *marginal*, not conditional — on the worst single machine in the evaluation it was 90.5% against a nominal 95%, so an outlier machine can breach the stated level.

Redact before a plan leaves the machine

`EXPLAIN` output embeds your query's literal values. On a real database those are user data.

```
cetp redact plan.json -o plan.redacted.json          # strips literals, keeps structure
cetp redact plan.json -o plan.pub.json --level full  # also hashes identifiers
cetp redact plan.redacted.json --check               # verify; exits 1 if anything survives
```

`structural` keeps relation and column names (fine for a first-party service, since schema names can still be revealing). `full` hashes them too, which is what publishing needs.

Register a machine

You cannot predict for a machine without its signature.

```
cetp signature list                                  # what is known
cetp signature probe --machine-id prod-db-1 --db mydb
```

The probe takes about 40 seconds and measures seven axes. It probes compute at the **achievable degree of parallelism** (`1 + max_parallel_workers_per_gather`), not at the core count — a query can never use more than that, and measuring across every core overstates capacity by 1.69× on an 8-core machine, which cost 324% vs 153% MAPE in testing.

Two things to know:

- The five built-in `legacy` machines (c5a, m5a, r5n, z1d, c7i) use a frozen signature whose bandwidth probe **no longer exists**. You cannot add a machine to that scale, and `cetp` will refuse to compare a legacy machine with a newly probed one.
- The signature's two axes correlate at **r = +0.958** on the reference machines, so treat it as one capability axis, not two.

CI

`.github/workflows/cetp-regression-check.yml` runs the regression check on pull requests that touch `.sql` files, redacts the plans before uploading them, and writes a summary. Point the `Load schema and fixture data` step at however your project materialises a representative dataset — a regression check means nothing if the data differs between the two runs.

What this does not do

- **Does not replace a staging benchmark** for an absolute SLA decision. It tells you when it does not know, and you should believe it.
- **Does not model concurrency.** All calibration is single-query-at-a-time; measured concurrency elasticity ranges from +0.286 to +0.984 across queries, so no single factor covers it.
- **Does not model PostgreSQL version.** Version alone shifts runtime ~30% and changes 12 of 22 plans on identical hardware and data.
- **Is calibrated on 5 machines × 21 TPC-H queries at SF1**, all EBS-backed. Outside that envelope the bounds are extrapolation.

Honest accuracy summary, leave-one-machine-out: analytical roofline **21.61%** MAPE, `1/compute_ratio` **20.06%**, bandwidth-ratio baseline **23.05%**. Full numbers in results/CANONICAL.md.